

appropriate format for later processing by the loader, and print an assembly listing containing the original source and the hexadecimal equivalent of the bytes generated. The LC is initialized as in pass 1, and the processing continues as follows.

A card is read from the source file left by pass 1. As in pass 1, the op code field is examined to determine if it is a pseudo op; if it is not, the MOT is searched to find a match for the card's op code field. The matching MOT entry specifies the length, binary op code, and the format type of the instruction. The operand fields of the different instruction format types require somewhat different processing.

For the RR-format instructions, each of the two register specification fields is evaluated. This evaluation may be very simple, as in: AR 2,3 or more complex, as in: MR EVEN, EVEN+1

The two fields are inserted into their respective four bit fields in the second byte of the RR instruction.

For RX format instruction, the register and index fields are evaluated and processed in the same way as the register specifications for RR format instructions, the storage address operand is evaluated to generate an Effective Address (EA). Then the BT must be examined to find a suitable base register (B) such that $D=EA-B(C)<4096$. The corresponding displacement can be determined. The 4 bit base register specification and 12 bit displacement field are then assembled into the third and fourth bytes of the instruction. Only the RR and RX instruction type are explicitly shown in the flowchart, the other instruction formats are handled similarly.

After the instruction has been assembled, it is put into the necessary format for later processing by the loader. Typically, several instructions are placed on single card. A listing line containing a copy of the source card, its assigned storage location, and its hexadecimal representation is then printed. Finally, the LC is incremented and processing is continued with the next card.

As in pass 1, each of the pseudo ops call for special processing. The EQU pseudo op requires very little processing in pass 2, because symbol definition was completed in pass 1. It is necessary only to print the EQU card as part of the printed listing.

The USING and DROP pseudo ops, which were largely ignored in pass 1, require additional processing in pass 2. The operand fields of the pseudo ops are evaluated then the corresponding BT entry is either marked as available, if USING, or unavailable, if DROP. The BT is used extensively in pass 2 to compute the base and displacement fields for machine instructions with storage operands.

The DS and DC pseudo ops are processed essentially as in pass 1. In pass 2, however, actual code must be generated for the DC pseudo op. depending upon the data types specified, this involves various conversions (e.g. floating point character to binary representation) and symbol evaluations (e.g. address constants).

The END pseudo ops indicate the end of the source program and terminates the assembly. Various “housekeeping” tasks must be generated for any literal remaining on the LT.

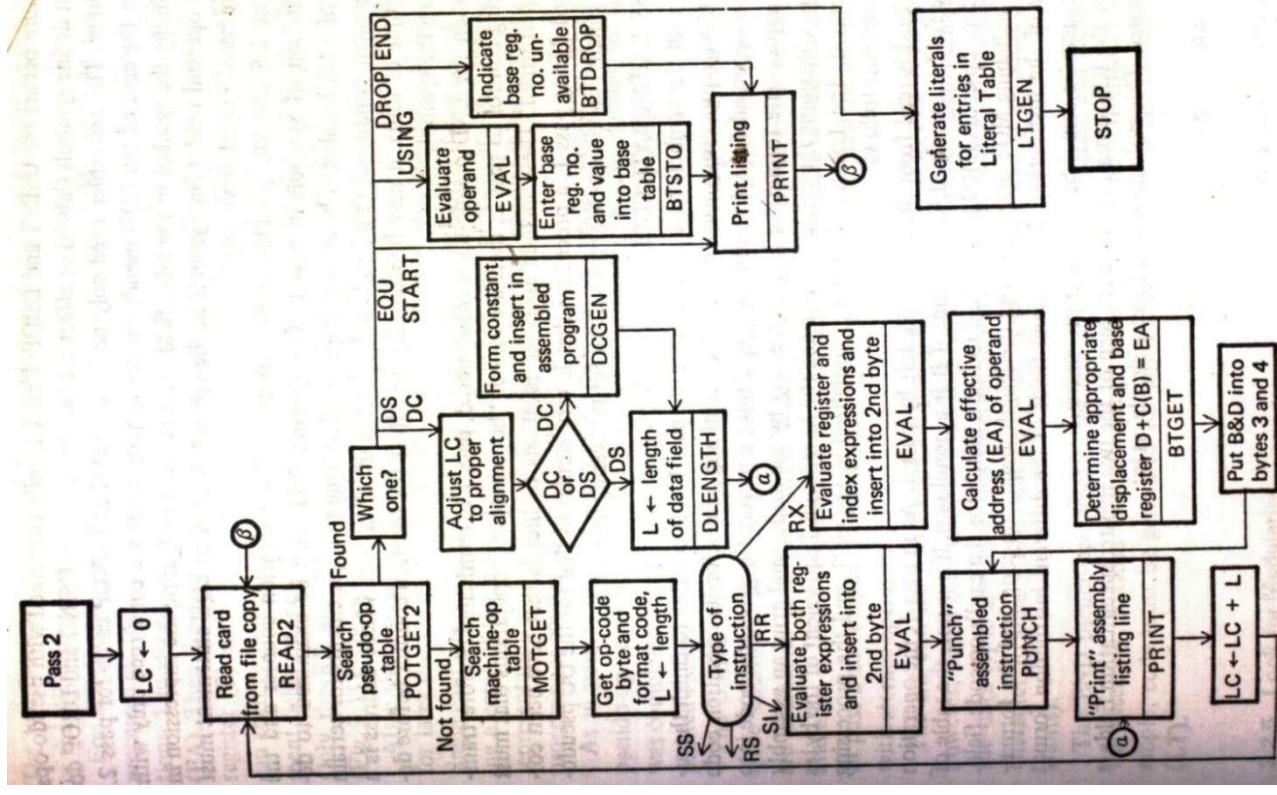


FIGURE 3.11 Detailed pass 2 flowchart